

1. Structure Initiale (Axée Frontend)

Au début, l'essentiel de votre travail sera dans la partie visible par l'utilisateur. Vous créerez des pages statiques (HTML/CSS) ou utiliserez un framework Frontend (comme React, Vue, si vous ne comptez pas utiliser les templates de Python).

```
mon_projet_appli_web/
├── frontend/                                # NOUVEAU dossier pour isoler le code de
└── l'interface
    ├── index.html                          # Page d'accueil principale
    ├── css/                               # Styles de l'application
    │   └── style.css
    ├── js/                                # Fichiers JavaScript côté client
    │   (interactions, appels API simulés)
    │   ├── main.js
    │   └── assets/                         # Images, icônes, polices
├── backend/                               # Dossier pour le code Python (laissée
│   vide pour l'instant)
├── requirements.txt
└── README.md
```

Objectif de cette phase :

1. **Maquettes** : Définir l'apparence et l'ergonomie.
2. **Maquette d'API (Mocking)** : Simuler les appels à l'API dans le `js/main.js` pour que le Frontend puisse "faire semblant" de recevoir des données. Cela permet de développer l'interface utilisateur sans attendre que l'API Python soit prête.

2. Intégration Progressive du Backend (Python)

Une fois que votre interface est stable, vous pouvez ajouter le cœur Python, l'API et la base de données. Le dossier `backend/` prendra la structure discutée précédemment.

Étape 2.1 : Mise en Place du Serveur Python et de l'API

Vous commencez par ajouter le point d'entrée du serveur et la logique de routing pour recevoir les requêtes du Frontend.

```
mon_projet_appli_web/
├── frontend/
├── ...
├── backend/
│   ├── api/                               # Contient les définitions des routes API
│   │   ├── users.py                       # Routes pour la gestion des utilisateurs
│   │   └── posts.py                       # Routes pour les publications
│   ├── core/                             # Fichiers d'initialisation (BD,
│   │   configuration)
│   ├── main.py                           # POINT D'ENTRÉE de l'application
│   │   (Flask/FastAPI/Django)
│   └── __init__.py
├── ...
```

Rôle des éléments Python :

- **backend/main.py** : Lance le serveur web (ex. : `app.run()`).
- **backend/api/** : Remplace les données simulées (mock data) de votre `frontend/js/main.js` par de vraies routes HTTP qui peuvent retourner des données en format **JSON**.

Étape 2.2 : Ajout de la Base de Données (BD) et de la Logique Métier

Vous ajoutez maintenant les couches qui permettent de sauvegarder et de manipuler réellement les données.

```
mon_projet_appli_web/
├── frontend/
...
├── backend/
│   ├── api/
│   └── models/                                # Les structures de données (tables de la
BD)
│       ├── User.py
│       └── services/                          # La logique métier (interaction avec la
BD)
│           ├── user_service.py
│           ├── core/
│           ├── main.py
│           └── __init__.py
...
```

Flux des données (Backend) :

1. Une requête arrive sur une **Route API** (ex: `/api/users`).
2. La fonction de la route appelle le **Service** approprié (`user_service.get_all_users()`).
3. Le **Service** utilise le **Modèle** (`User.py`) pour interroger la base de données.
4. Les données sont renvoyées par le Service à l'API, qui les formate en JSON et les envoie au Frontend.

Étape 2.3 : Liaison Finale

La dernière étape est de remplacer dans votre `frontend/js/main.js` les appels aux données simulées par de vrais appels HTTP vers l'API Python (qui elle-même interagit maintenant avec la BD).

Cette approche progressive est la méthode **Frontend-First** ou **API-Design First** : elle assure que l'interface utilisateur est fonctionnelle et que l'API répond exactement aux besoins de cette interface avant d'écrire la logique interne.